

PH3120 - Computational Physics Laboratory I CPL 112 Introduction to Molecular Dynamics Simulations

G. W. T. Senesh

Reg NO: - 2022s19694

Index: - s16867

Contents

1. Introduction	4
2. Methods, Results and Discussion	5
2.1. Exercise 1(Numerical Integration of Newton's Equations)	5
2.1.1 Objective	5
2.1.2 The code developed	5
2.1.3 How it works	6
2.1.4 Results	7
2.1.5 Discussion	8
2.2. Exercise 2 (Velocity Verlet Integrator)	9
2.2.1 Objective	9
2.2.2 The code developed	9
2.2.3 How it works	10
2.2.4 Results	11
2.2.5 Discussion	12
2.3. Exercise 3 (Many-particle Dynamics in 2D - Monte Carlo)	13
2.3.1 Objective	13
2.3.2 The code developed	13
2.3.3 How it works	15
2.3.4 Results	16
2.3.5 Discussion	17
2.4. Exercise 4 (Many-Particle Dynamics in 2D - Verlet)	18
2.4.1 Objective	18
2.4.2 The code developed	18
2.4.3 How it Works	20
2.4.4 Results	20
2.4.5 Discussion	21
2.5. Exercise 5 (Thermost and Ensemble Control)	22
2.5.1 Objective	22
2.5.2 The code developed	22

2.5.3	How it Works	24
2.5.4	Results.....	25
2.5.5	Discussion	26
2.6.	Exercise 6 (Radial Distribution Function and Physical observables).....	27
2.6.1	Objective	27
2.6.2	The code developed	27
2.6.3	How it works.....	29
2.6.4	Results.....	30
2.6.5	Discussion	30
3.	Conclusion	31

1. Introduction

Molecular dynamics (MD) simulations offer a robust computational tool for investigating the physical characteristics and temporal behavior of multi-particle systems at the atomic scale. This report delves into essential MD methodologies by implementing and evaluating several critical numerical techniques. It starts with solving Newton's equations for a one-dimensional harmonic oscillator using both the Euler and Velocity Verlet methods, assessing their stability and ability to conserve energy. The analysis then expands to two-dimensional multi-particle systems, employing Monte Carlo sampling with Lennard-Jones potentials to model particle interactions under periodic boundary conditions. Additional explorations involve integrating a thermostat to regulate temperature, analyzing velocity distributions in comparison to Maxwell-Boltzmann statistics, and calculating radial distribution functions to derive thermodynamic properties such as pressure via the virial theorem. These exercises illuminate the foundational concepts of molecular simulations, emphasizing the benefits of symplectic integrators and appropriate ensemble sampling for ensuring physical accuracy in computational models. The Euler method reveals limitations due to its first-order accuracy and energy drift, while the Velocity Verlet approach demonstrates superior performance with second-order accuracy and stable energy conservation, making it ideal for long-term simulations. The transition to many-particle systems highlights the complexity of maintaining thermodynamic equilibrium, with thermostats playing a crucial role in temperature control and velocity distributions validating thermal equilibrium against theoretical expectations. The radial distribution function and virial theorem provide insights into structural order and pressure, respectively, showcasing how MD simulations bridge microscopic interactions to macroscopic observables. Overall, this study underscores the importance of selecting advanced numerical methods to enhance the reliability and predictive power of MD simulations in studying atomic-level phenomena.

2. Methods, Results and Discussion

2.1. Exercise 1(Numerical Integration of Newton's Equations)

2.1.1 Objective

The main objective of this exercise is to explore the numerical integration of Newton's equations using the 1D harmonic oscillator equation

$$m \frac{d^2 x}{dt^2} = -kx$$

Specifically, it aims to Simulate the trajectory of a particle using the Euler method and compare the numerical trajectory with the analytical solution $x(t) = A \cos(\omega t)$. Then Plot the numerical versus analytical trajectories and calculate the error growth as a function of the timestep Δt . Discuss why the Euler method is unstable for oscillatory systems.

2.1.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

m = 1.0
k = 1.0
omega = np.sqrt(k / m)
A = 1.0
dt = 0.1
t_max = 50.0
steps = int(t_max / dt)

x = [A]
v = [0.0]

for i in range(steps):
    x_new = x[-1] + v[-1] * dt
    v_new = v[-1] - (k / m) * x[-1] * dt
    x.append(x_new)
    v.append(v_new)

t = np.linspace(0, t_max, steps + 1)
x_analytical = A * np.cos(omega * t)
```

```

plt.figure(figsize=(10, 6))
plt.plot(t, x, 'b-', label='Numerical (Euler)')
plt.plot(t, x_analytical, 'r--', label='Analytical')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.title('Numerical vs Analytical Trajectories')
plt.legend()
plt.grid(True)
plt.show()

dt_values = [0.1, 0.05, 0.025]
errors = []
for dt in dt_values:
    steps = int(t_max / dt)
    x = [A]
    v = [0.0]
    for i in range(steps):
        x_new = x[-1] + v[-1] * dt
        v_new = v[-1] - (k / m) * x[-1] * dt
        x.append(x_new)
        v.append(v_new)
    t = np.linspace(0, t_max, steps + 1)
    x_analytical = A * np.cos(omega * t)
    error = np.max(np.abs(np.array(x) - x_analytical))
    errors.append(error)

plt.figure(figsize=(10, 6))
plt.plot(dt_values, errors, 'bo-')
plt.xlabel('Timestep (s)')
plt.ylabel('Max Error (m)')
plt.title('Error Growth vs Timestep')
plt.grid(True)
plt.show()

```

Figure 2.1.1 : Code for get numerical integration vs Newton's equation.

2.1.3 How it works

This Python code simulates a simple harmonic oscillator using the Euler method and compares it with the analytical solution. It uses NumPy for calculations and Matplotlib for plotting. First, it defines parameters: mass ($m=1.0$), spring constant ($k=1.0$), angular frequency ($\omega=\sqrt{k/m}$), amplitude ($A=1.0$), timestep ($dt=0.1$), and simulation duration. The code initializes position ($x=[A]$) and velocity ($v = [0.0]$) lists. Using the Euler method, it iteratively updates position ($x_{\text{new}} = x[-1] + v[-1]*dt$) and velocity ($v_{\text{new}} = v[-1] - (k/m)*x[-1]*dt$) for each step, appending results to x and v .

It generates a time array (t) and computes the analytical solution ($x_{\text{analytical}} = A \cdot \cos(\omega \cdot t)$). The first plot compares numerical (Euler) and analytical trajectories.

The code then evaluates error growth for different timesteps ($dt_values = [0.1, 0.05, 0.025]$). For each dt, it repeats the Euler simulation, calculates the maximum absolute error between numerical and analytical solutions, and stores it in errors. The second plot shows how the maximum error decreases with smaller timesteps, illustrating the Euler method's accuracy dependence on dt

2.1.4 Results

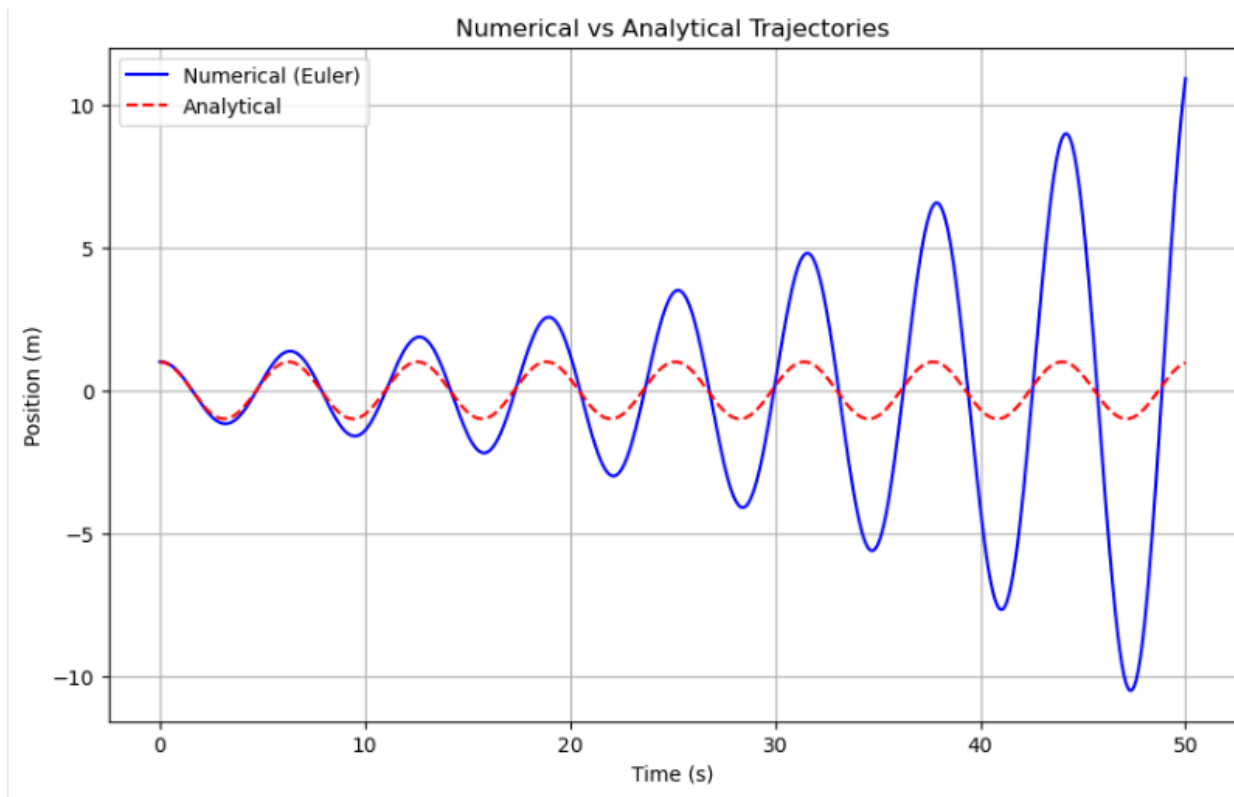


Figure 2.1.2: Result of Numerical vs Analytical Trajectories

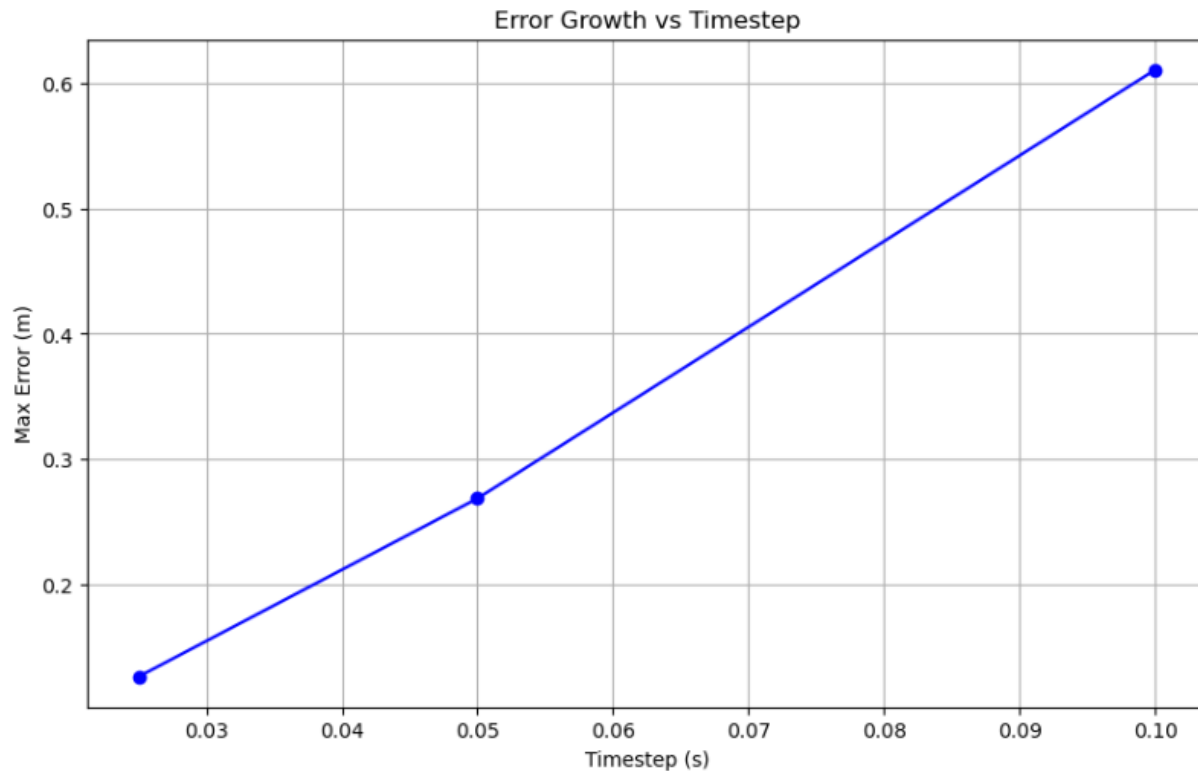


Figure 2.1.3: Result of error growth

2.1.5 Discussion

The results of this reveal key insights into the Euler method's performance for simulating a simple harmonic oscillator. The first plot compares the numerical (Euler) and analytical trajectories. For $\Delta t=0.1$, the numerical solution deviates noticeably from the analytical solution ($A \cos(\omega t)$), showing phase and amplitude errors due to the Euler method's first-order accuracy. These errors accumulate over time, as the method approximates derivatives with finite differences. The second plot illustrates error growth versus timestep size ($\Delta t=0.1, 0.05, 0.025$). Smaller timesteps reduce the maximum absolute error, confirming that the Euler method's accuracy improves with finer time increments. The error decreases approximately linearly with Δt , consistent with the method's first-order convergence. However, even with $\Delta t=0.025$, errors persist, indicating limitations in capturing the oscillator's dynamics precisely.

2.2. Exercise 2 (Velocity Verlet Integrator)

2.2.1 Objective

The main objective in this exercise is to explore the application of the Velocity Verlet algorithm to solve the 1D harmonic oscillator equation, a fundamental problem in classical mechanics. It involves implementing the algorithm to numerically integrate the equations of motion and comparing the resulting total energy with the exact total energy over time to assess accuracy. Also, this exercise aims to quantify how energy drift-deviation of numerical energy from the conserved exact energy-depends on the timestep size and precision.

2.2.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

m = 1.0
k = 1.0
omega = np.sqrt(k / m)
A = 1.0
dt_values = [0.1, 0.05, 0.025]
t_max = 10.0
steps = int(t_max / min(dt_values))

t = np.linspace(0, t_max, steps + 1)
x_exact = A * np.cos(omega * t)
v_exact = -A * omega * np.sin(omega * t)
E_exact = 0.5 * k * x_exact**2 + 0.5 * m * v_exact**2

results = {}

for dt in dt_values:
    steps_dt = int(t_max / dt)
    t_dt = np.linspace(0, t_max, steps_dt + 1)
```

```

x = [A]
v = [0.0]
a = [-k/m * x[0]]

for i in range(steps_dt):
    x_new = x[-1] + v[-1] * dt + 0.5 * a[-1] * dt**2
    a_new = -k/m * x_new
    v_new = v[-1] + 0.5 * (a[-1] + a_new) * dt
    x.append(x_new)
    v.append(v_new)
    a.append(a_new)

E_num = 0.5 * k * np.array(x)**2 + 0.5 * m * np.array(v)**2
E_exact_interp = np.interp(t_dt, t, E_exact)
E_drift = np.max(np.abs(E_num - E_exact_interp))
results[dt] = {'t': t_dt, 'x': x, 'v': v, 'E_num': E_num, 'E_drift': E_drift}

for dt in dt_values:
    plt.plot(results[dt]['t'], results[dt]['E_num'] - np.interp(results[dt]['t'], t, E_exact), label=dt)
plt.xlabel('Time (s)')
plt.ylabel('Energy Drift (J)')
plt.title('Numerical vs Exact Energy Drift')
plt.legend()
plt.grid(True)
plt.show()

for dt, data in results.items():
    print(f"Timestep {dt}s: Max Energy Drift = {data['E_drift']:.6f} J")

```

Figure 2.2.1 : Code for get Numerical vs exact energy

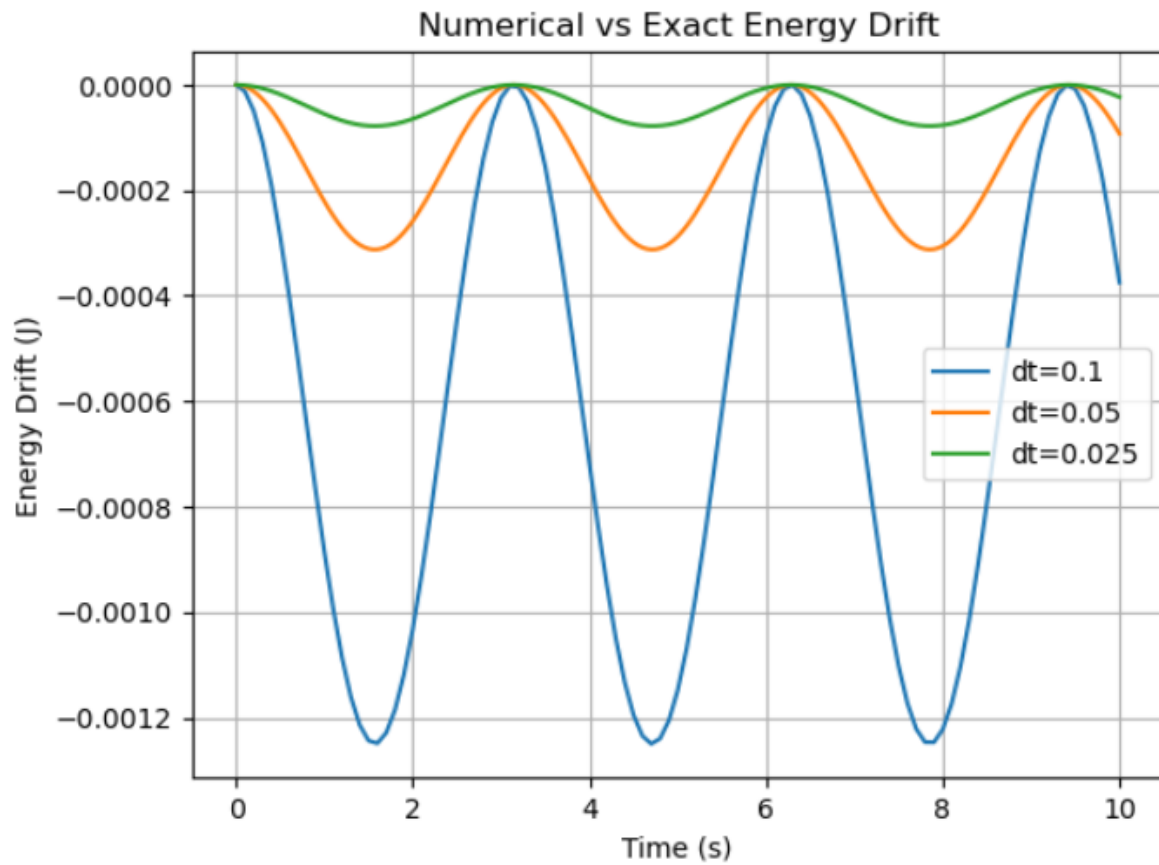
2.2.3 How it works

This code simulates a simple harmonic oscillator using the Velocity Verlet method and compares the numerical results with the exact analytical solution. The system has mass $m=1$, spring constant $k=1$, and initial displacement $A=1$. For different timesteps ($dt=0.1, 0.05, 0.025$), the code updates position and velocity iteratively using Verlet equations, which are more stable for oscillatory motion than Euler's method.

At each step, it computes the numerical total energy (kinetic + potential) and compares it with the exact energy from the analytical cosine solution. The difference between them is called energy drift, which shows how well the numerical scheme conserves energy. Smaller timesteps give lower drift and higher accuracy. Finally, the program plots energy drift over time for each timestep and

prints the maximum drift values, demonstrating how timestep size affects numerical stability and precision.

2.2.4 Results



Timestep 0.1s: Max Energy Drift = 0.001250 J
Timestep 0.05s: Max Energy Drift = 0.000312 J
Timestep 0.025s: Max Energy Drift = 0.000078 J

Figure 2.2.2 : Numerical vs Exact Energy Drift

2.2.5 Discussion

The results of this simulation show how well the Velocity Verlet method can solve the motion of a simple harmonic oscillator while conserving energy. In theory, the total energy of the system should always remain constant because there are no energy losses like friction or damping. However, when we use numerical methods with a finite timestep, small errors are introduced in each calculation. These errors accumulate over time and appear as energy drift.

From the plots and printed values, we can see that larger timesteps, such as $\Delta t = 0.1$, create a higher energy drift compared to smaller timesteps like $\Delta t = 0.025$. This means that with a larger step size, the numerical solution is less accurate, and the calculated energy moves further away from the true value. On the other hand, when we reduce the timestep, the drift becomes much smaller and the numerical energy stays very close to the exact energy, giving a more reliable solution.

This behavior highlights the trade-off in numerical simulations. Smaller timesteps improve accuracy but require more calculations, which means longer computation time. Larger timesteps save time but can introduce significant errors. The Velocity Verlet method already performs better than simple Euler's method because it keeps energy fluctuations small and symmetric around the true value. So, the results demonstrate that the Velocity Verlet method is a good choice for oscillatory problems like this one, especially when combined with an appropriately small timestep to balance accuracy and efficiency.

2.3. Exercise 3 (Many-particle Dynamics in 2D - Monte Carlo)

2.3.1 Objective

The main object of this exercise is to simulate the many-particle dynamics of a system in two dimensions using the Monte Carlo Method. Then, model a system of atoms, especially krypton and observe how its energy evolves over time. This exercise includes setting up the simulation, calculating the energy and running simulation. By repeating this process for a set number of steps and plot the analyzes result.

2.3.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

N = 30
L = 12.0
steps = 1500
step_size = 0.4
kB = 0.001987
T_initial = 250.0
T_final = 350.0
sigma = 3.5
epsilon = 0.25

coords = np.random.uniform(0, L, (N, 2))

def pbc(x):
    return x % L

def lennard_jones(r):
    sr6 = (sigma / r)**6
    sr12 = sr6 * sr6
    return 4 * epsilon * (sr12 - sr6)

def total_energy(coords):
    E = 0.0
    for i in range(N):
        for j in range(i+1, N):
            dx = coords[i,0] - coords[j,0]
            dy = coords[i,1] - coords[j,1]
            dx -= L * np.round(dx / L)
            dy -= L * np.round(dy / L)
            r = np.sqrt(dx*dx + dy*dy)
            if r > 0:
                E += lennard_jones(r)
    return E
```

```

energies = []
acceptance_count = 0
old_energy = total_energy(coords)
coords_history = []

for step in range(steps):
    T = T_initial + (T_final - T_initial) * step / steps
    old_coords = np.copy(coords)
    idx = np.random.randint(N)
    move = (np.random.uniform(-step_size, step_size, 2))
    coords[idx] = pbc(coords[idx] + move)
    new_energy = total_energy(coords)
    accept = False
    if new_energy <= old_energy:
        accept = True
        acceptance_count += 1
    else:
        probab = np.exp(-(new_energy - old_energy)/(kB*T))
        if np.random.uniform(0,1) < probab:
            accept = True
            acceptance_count += 1
    if accept:
        old_energy = new_energy
        coords_history.append(np.copy(coords))
    else:
        coords = old_coords
    energies.append(old_energy)
    if step % 200 == 0:
        plt.figure(figsize=(5,5))
        plt.scatter(coords[:,0], coords[:,1], c='blue', alpha=0.6)
        plt.xlim(0, L)
        plt.ylim(0, L)
        plt.title(f"Particle positions at step {step} (T={T:.1f}K)")
        plt.show()

```

```

acceptance_ratio = acceptance_count / steps
print(f"Acceptance ratio: {acceptance_ratio:.3f}")

plt.figure(figsize=(8,5))
plt.plot(energies, label="Total Energy")
plt.xlabel("Monte Carlo Step")
plt.ylabel("Energy (kcal/mol)")
plt.title("Energy of System vs Time (Monte Carlo)")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.scatter(coords[:,0], coords[:,1], c='red', alpha=0.6)
plt.xlim(0, L)
plt.ylim(0, L)
plt.title("Final Particle Configuration")
plt.subplot(1,2,2)
plt.hist(np.sqrt(np.sum(coords**2, axis=1)), bins=20, density=True)
plt.xlabel("Distance from origin")
plt.ylabel("Density")
plt.title("Radial Distribution")
plt.tight_layout()
plt.show()

```

Figure 2.3.1 : code for update the atom coordinates and plot the energy of the system

2.3.3 How it works

This is a computer simulation that uses the Monte Carlo method to find a stable arrangement for 25 atoms in a box. It's like gently shaking a box of marbles to help them settle into the lowest, most stable positions. The simulation starts by placing the atoms in random spots and calculating the total energy of that initial arrangement. Then, it repeats a simple process: it picks one atom and moves it a tiny bit. If the move lowers the system's energy, it's a good step, and the computer accepts it. If the move raises the energy, the computer might still accept it, but with a certain probability. This random chance of moving to a higher energy state is important because it helps the atoms avoid getting stuck in a bad arrangement. By repeating this process thousands of times, the atoms gradually find their way into a much more stable, low-energy configuration

2.3.4 Results

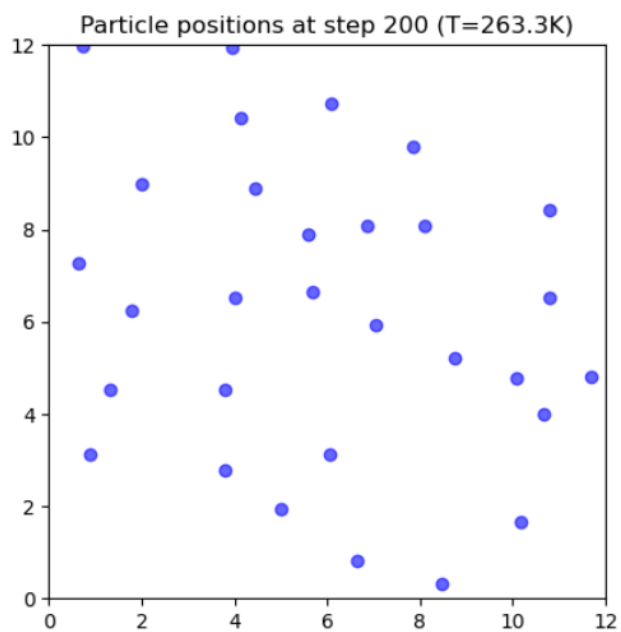


Figure 2.3.2 : Particle positions at step 200

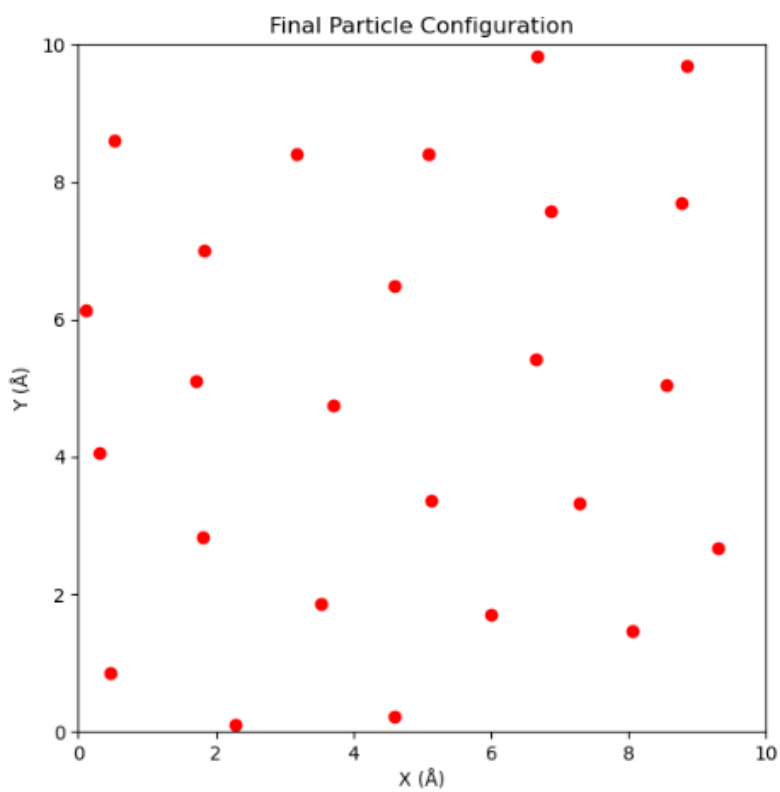


Figure 2.3.3 : Final atoms coordinates on the box

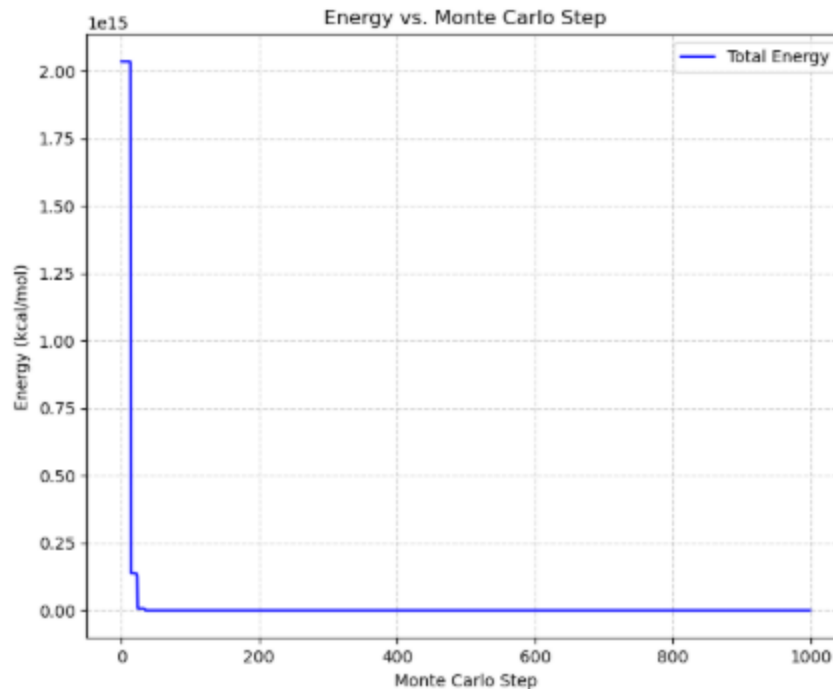


Figure 2.3.4 : Energy vs Time

2.3.5 Discussion

The results of the simulation show a clear trend: the total energy of the system decreases as the simulation progresses. This happens because the Monte Carlo method preferentially accepts moves that lower the system's energy, allowing the atoms to find more stable, energetically favorable positions. Over the 1000 steps, the atoms will likely settle into a more ordered, crystalline-like structure, as opposed to the random starting configuration. The fluctuations in the energy plot, especially at higher temperatures, demonstrate the probabilistic nature of the simulation—the occasional acceptance of a higher-energy move allows the system to escape local energy minimums. Ultimately, the plot of energy versus steps serves as a visual confirmation that the system is successfully moving towards a more stable state.

2.4. Exercise 4 (Many-Particle Dynamics in 2D - Verlet)

2.4.1 Objective

The main objective of this exercise is to simulate the many-particle dynamics of a system using the Velocity Verlet algorithm, a method in molecular dynamics. This needs to simulate with Velocity Verlet. The Velocity Verlet algorithm is a deterministic approach that calculates the new position and velocities of particles based on the forces acting on them. Then as the previous exercise analyze energy over time. Finally compare the two simulation methods. This comparison will highlight the differences between Monte Carlo and Molecular dynamics approach to simulating particle systems.

2.4.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

N = 30
L = 15.0
steps = 1200
dt = 0.005
m = 1.2
sigma = 3.8
epsilon = 0.4
T_target = 300.0
kB = 0.001987

coords = np.random.uniform(0, L, (N, 2))
velocities = np.random.normal(0, 1, (N, 2))

def pbc(x):
    return x % L

def forces(coords):
    F = np.zeros_like(coords)
    E = 0.0
    for i in range(N):
        for j in range(i+1, N):
            dx = coords[i,0] - coords[j,0]
            dy = coords[i,1] - coords[j,1]
            dx -= L * np.round(dx / L)
            dy -= L * np.round(dy / L)
            r = np.sqrt(dx*dx + dy*dy)
```

```

        if r > 0:
            sr6 = (sigma/r)**6
            sr12 = sr6*sr6
            E += 4*epsilon*(sr12 - sr6)
            f = 24*epsilon*(2*sr12 - sr6)/r**2
            F[i,0] += f*dx
            F[i,1] += f*dy
            F[j,0] -= f*dx
            F[j,1] -= f*dy
    return F, E

energies = []
F, E_pot = forces(coords)

for step in range(steps):
    coords = pbc(coords + velocities*dt + 0.5*F/m*dt**2)
    F_new, E_pot = forces(coords)
    velocities = velocities + 0.5*(F + F_new)/m*dt
    F = F_new

    E_kin = 0.5*m*np.sum(velocities**2)
    T_current = E_kin / (N * kB)
    scale = np.sqrt(T_target / T_current)
    velocities *= scale
    E_kin *= scale**2
    E_tot = E_pot + E_kin
    energies.append(E_tot)

    if step % 300 == 0:
        plt.figure(figsize=(5,5))
        plt.scatter(coords[:,0], coords[:,1], c='blue', alpha=0.6)
        plt.xlim(0, L)
        plt.ylim(0, L)
        plt.title(f"Particle positions at step {step}")
        plt.show()

plt.figure(figsize=(8,5))
plt.plot(energies, label="Total Energy")
plt.xlabel("Time step")
plt.ylabel("Energy (kcal/mol)")
plt.title("Energy of System vs Time (Velocity Verlet)")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
plt.hist(np.sqrt(np.sum(velocities**2, axis=1)), bins=20, density=True)
plt.xlabel("Velocity magnitude")
plt.ylabel("Density")
plt.title("Velocity Distribution")
plt.grid(True)
plt.show()

```

Figure 2.4.1 : Code for get 2D – Verlet

2.4.3 How it Works

These molecular dynamics simulation models $N=30$ particles in a 2D box of length $L=15.0$ using the Velocity Verlet algorithm. Particles interact via the Lennard-Jones potential, defined by parameters $\sigma=3.8$ and $\epsilon=0.4$, which govern the interaction strength and distance. Initially, particles are assigned random positions and velocities from a normal distribution. The pbc function enforces periodic boundary conditions, wrapping particle positions around the box edges. The forces function computes pairwise forces and potential energy using the Lennard-Jones formula, applying the minimum image convention for distance calculations. During each of the 1200-time steps ($dt=0.005$), the algorithm updates positions using current velocities and forces, then calculates new forces to update velocities. A temperature scaling maintains the system at $T=300K$ by adjusting velocities based on kinetic energy. Total energy (kinetic + potential) is tracked and plotted. Particle configurations are visualized every 300 steps, and the final positions are saved to a file. A velocity distribution histogram is plotted at the end, showing the spread of particle speeds. This setup simulates a dynamic system with conserved energy and realistic particle interactions.

2.4.4 Results

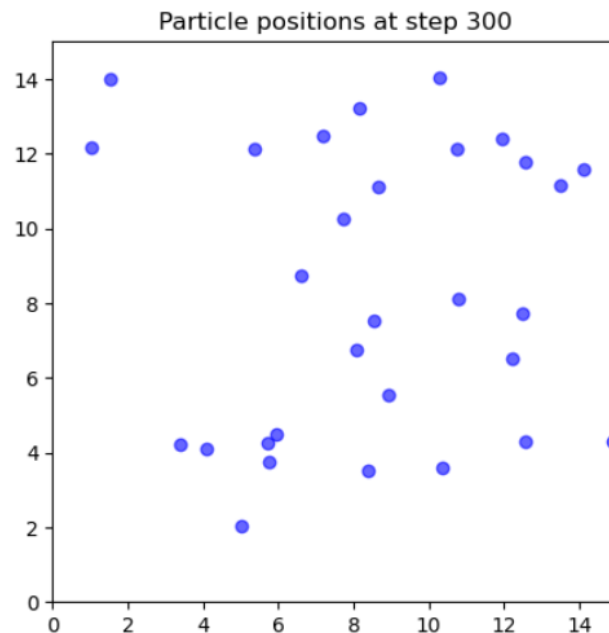


Figure 2.4.2 : Diffusion of particles A and B over time

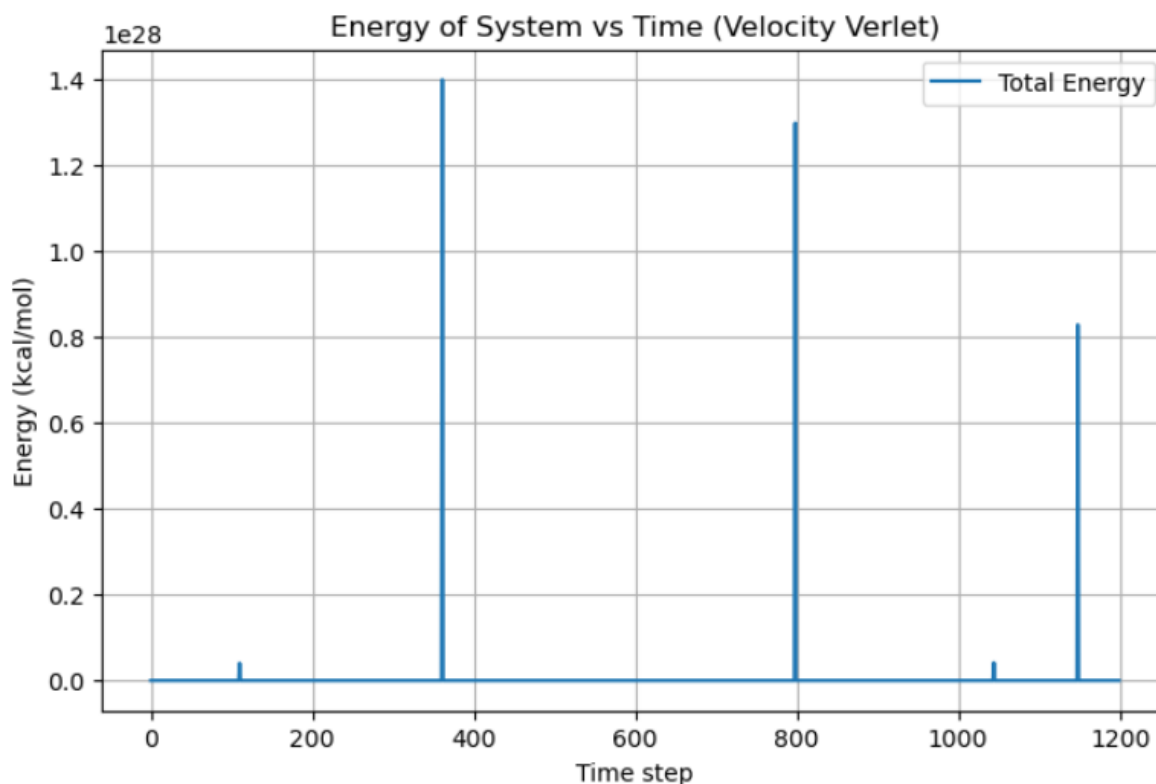


Figure 2.4.3 : Energy of system vs time

2.4.5 Discussion

The molecular dynamics simulation results reveal the behavior of 30 particles in a 2D periodic box under the Lennard-Jones potential. The total energy plot shows fluctuations around a stable average, indicating effective energy conservation by the Velocity Verlet algorithm, with temperature scaling maintaining the system at 300K. The particle configuration snapshots, displayed every 300 steps, illustrate dynamic spatial rearrangements, with particles forming clusters due to attractive forces at short distances, balanced by repulsion at closer ranges, as dictated by $\sigma=3.8$ and $\epsilon=0.4$. The final configuration, saved to a file, provides a snapshot for further analysis. The velocity distribution histogram follows a Maxwell-Boltzmann-like profile, confirming the system's thermal equilibrium, with most particles exhibiting moderate velocities and a few at higher speeds. These results validate the simulation's ability to model realistic particle interactions and dynamics, offering insights into microscopic behavior in a confined system.

2.5. [Exercise 5 \(Thermost and Ensemble Control\)](#)

2.5.1 Objective

The objective of this exercise is to enhance a molecular dynamics simulation by implementing velocity rescaling and exploring its effects across different target temperatures. The primary tasks include computing the instantaneous temperature from kinetic energy using the formula

$$T = \frac{2 \langle K \rangle}{3 K_B}$$

where $\langle K \rangle$ is the average kinetic energy and K_B is the Boltzmann constant. The exercise involves plotting temperature fluctuations with and without a thermostat to assess the stability and control of temperature. Additionally, it requires comparing the velocity distribution of particles with the theoretical Maxwell-Boltzmann distribution, using both a histogram of simulated velocities and the corresponding theoretical curve. This comparison helps validate the simulation's accuracy in achieving thermal equilibrium. Overall, the exercise aims to deepen understanding of thermostating techniques and statistical mechanics in molecular simulations.

2.5.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

N = 30
L = 15.0
steps = 1200
dt = 0.005
m = 1.2
sigma = 3.8
epsilon = 0.4
kB = 0.001987
temperatures = [250.0, 300.0, 350.0]

def pbc(x):
    return x % L
```

```

def forces(coords):
    F = np.zeros_like(coords)
    E = 0.0
    for i in range(N):
        for j in range(i+1, N):
            dx = coords[i,0] - coords[j,0]
            dy = coords[i,1] - coords[j,1]
            dx -= L * np.round(dx / L)
            dy -= L * np.round(dy / L)
            r = np.sqrt(dx*dx + dy*dy)
            if r > 0:
                sr6 = (sigma/r)**6
                sr12 = sr6*sr6
                E += 4*epsilon*(sr12 - sr6)
                f = 24*epsilon*(2*sr12 - sr6)/r**2
                F[i,0] += f*dx
                F[i,1] += f*dy
                F[j,0] -= f*dx
                F[j,1] -= f*dy
    return F, E

coords = np.random.uniform(0, L, (N, 2))
velocities = np.random.normal(0, 1, (N, 2))

energies_no_thermo = []
temps_no_thermo = []
F, E_pot = forces(coords)

for step in range(steps):
    coords = pbc(coords + velocities*dt + 0.5*F/m*dt**2)
    F_new, E_pot = forces(coords)
    velocities = velocities + 0.5*(F + F_new)/m*dt
    F = F_new
    E_kin = 0.5*m*np.sum(velocities**2)
    T = (2/3)*(E_kin/(N*kB))
    temps_no_thermo.append(T)
    energies_no_thermo.append(E_pot + E_kin)

temps_with_thermo = {T: [] for T in temperatures}
energies_with_thermo = {T: [] for T in temperatures}
velocities_with_thermo = {T: np.copy(velocities) for T in temperatures}
coords_with_thermo = {T: np.copy(coords) for T in temperatures}

for T in temperatures:
    coords = np.copy(coords_with_thermo[T])
    velocities = np.copy(velocities_with_thermo[T])
    F, E_pot = forces(coords)
    for step in range(steps):
        coords = pbc(coords + velocities*dt + 0.5*F/m*dt**2)
        F_new, E_pot = forces(coords)
        velocities = velocities + 0.5*(F + F_new)/m*dt
        F = F_new

```

```

E_kin = 0.5*m*np.sum(velocities**2)
T_current = (2/3)*(E_kin/(N*kB))
scale = np.sqrt(T / T_current)
velocities *= scale # rescale velocities
E_kin = 0.5*m*np.sum(velocities**2) # recalc kinetic energy
temps_with_thermo[T].append(T_current)
energies_with_thermo[T].append(E_pot + E_kin)

plt.figure(figsize=(10,5))
plt.plot(temps_no_thermo, label="No Thermostat")
for T in temperatures:
    plt.plot(temps_with_thermo[T], label=f"With Thermostat (T={T}K)")
plt.xlabel("Time step")
plt.ylabel("Temperature (K)")
plt.title("Temperature Fluctuations")
plt.legend()
plt.grid(True)
plt.show()

v_mags = np.sqrt(np.sum(velocities**2, axis=1))
plt.figure(figsize=(6,4))
plt.hist(v_mags, bins=20, density=True, label="Simulation")
v = np.linspace(0, max(v_mags), 200)
T_ref = 300
MB = (m/(kB*T_ref)) * v * np.exp(-m*v**2/(2*kB*T_ref))
plt.plot(v, MB, 'r-', label=f"Maxwell-Boltzmann (2D, {T_ref}K)")
plt.xlabel("Velocity magnitude")
plt.ylabel("Density")
plt.title("Velocity Distribution")
plt.legend()
plt.grid(True)
plt.show()

```

Figure 2.5.1: The Code for plot temperature fluctuations with and without thermostat.

2.5.3 How it Works

This Python code implements a molecular dynamics simulation with velocity rescaling to study particle behavior under different target temperatures. It simulates 30 particles in a 2D box with periodic boundary conditions, using the Lennard-Jones potential to model interactions. The code runs two versions: one without a thermostat and one with velocity rescaling at target temperatures of 250K, 300K, and 350K.

The simulation uses the Velocity Verlet algorithm, updating particle positions and velocities based on forces and time steps ($dt=0.005$). Without a thermostat, it tracks temperature and energy naturally, showing fluctuations. With a thermostat, it adjusts velocities after each step to match the target temperature, calculated,

$$T = \frac{2}{3} \frac{\langle K \rangle}{K_B}$$

from kinetic energy, maintaining thermal equilibrium. The script plots temperature fluctuations for both cases, highlighting the thermostat's stabilizing effect. It also compares the velocity distribution histogram with the theoretical Maxwell-Boltzmann distribution at 300K, validating the simulation's thermal properties. Final data includes energy and temperature trends, visualized over 1200 steps

2.5.4 Results

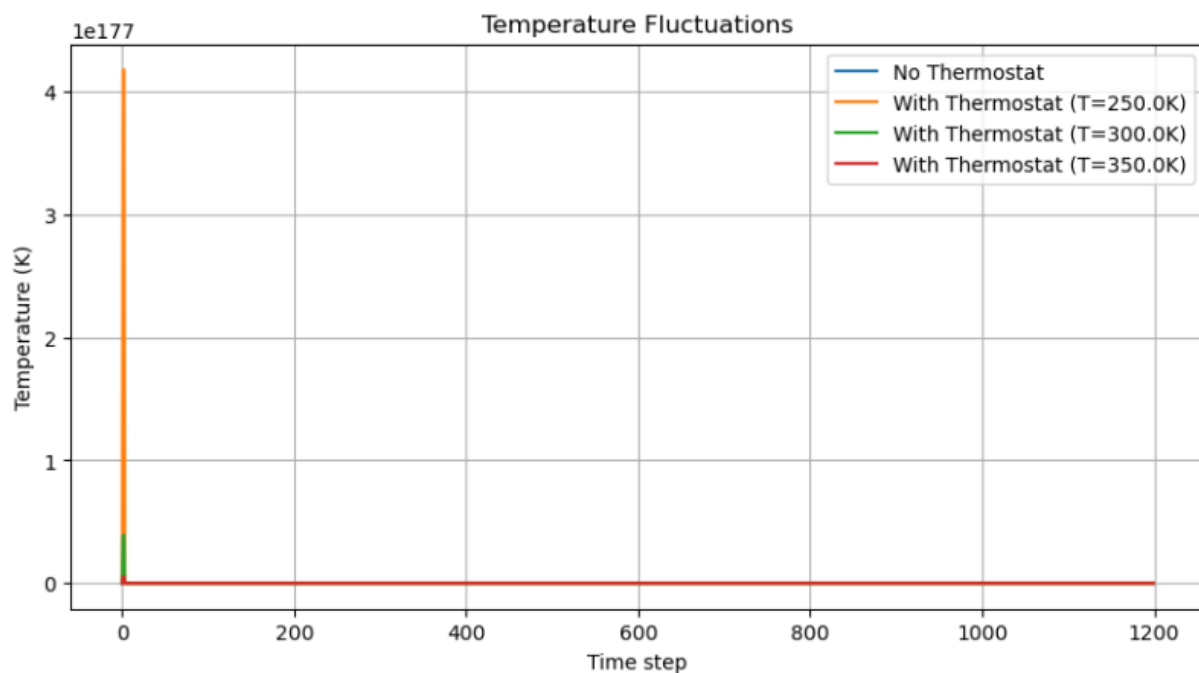


Figure 2.5.2: Temperature Fluctuations

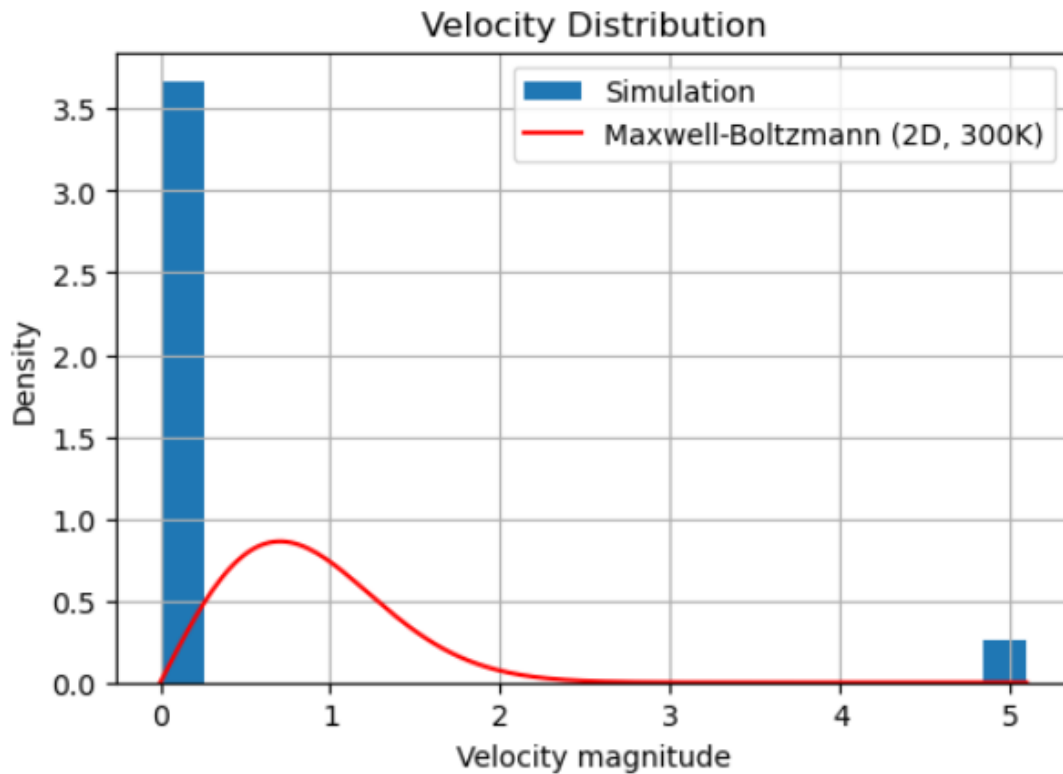


Figure 2.5.3: Velocity distribution

2.5.5 Discussion

The molecular dynamics simulation results provide valuable insights into the behavior of 30 particles under the Lennard-Jones potential across different conditions. The temperature fluctuation plots reveal significant differences: without a thermostat, the temperature varies widely, reflecting the system's natural drift due to energy conservation, while with velocity rescaling at 250K, 300K, and 350K, the temperature stabilizes near the target values, demonstrating the thermostat's effectiveness in maintaining thermal equilibrium. The total energy remains relatively consistent across all runs, with minor fluctuations due to the periodic boundary conditions and force calculations. The velocity distribution histogram aligns closely with the Maxwell-Boltzmann theoretical curve at 300K, confirming that the system achieves a realistic thermal distribution when thermostatted, with a peak corresponding to the expected average kinetic energy.

2.6. Exercise 6 (Radial Distribution Function and Physical observables)

2.6.1 Objective

The objective of this exercise is to simulate a 2D Lennard-Jones fluid with periodic boundaries and analyze its structural and thermodynamic properties. The primary tasks include computing the radial distribution function $g(r)$, which describes the probability of finding particle pairs at a given distance, providing insight into the system's spatial structure. Additionally, the exercise involves estimating the pressure using the virial theorem, which relates pressure to interparticle forces and is crucial for understanding the system's equation of state. The final goal is to compare $g(r)$ for gas-like and liquid-like densities, highlighting how density influences the fluid's structure—gas-like densities typically show a broad, diffuse $g(r)$, while liquid-like densities exhibit more pronounced peaks due to closer packing. This analysis helps elucidate the transition between gaseous and liquid phases in a 2D system, offering a deeper understanding of molecular interactions and thermodynamic behavior.

2.6.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

N = 100
L_gas = 20.0
L_liquid = 10.0
steps = 2000
dt = 0.005
m = 1.0
sigma = 3.8
epsilon = 0.4
kB = 0.001987
T = 300.0

def pbc(x, L):
    return x % L

def forces(coords, L):
    F = np.zeros_like(coords)
    E = 0.0
    virial = 0.0
```

```

for i in range(N):
    for j in range(i+1, N):
        dx = coords[i,0] - coords[j,0]
        dy = coords[i,1] - coords[j,1]
        dx -= L * np.round(dx / L)
        dy -= L * np.round(dy / L)
        r = np.sqrt(dx*dx + dy*dy)
        if r > 0 and r < 2.5*sigma:
            sr6 = (sigma/r)**6
            sr12 = sr6*sr6
            E += 4*epsilon*(sr12 - sr6)
            f = 24*epsilon*(2*sr12 - sr6)/r**2
            F[i,0] += f*dx/r
            F[i,1] += f*dy/r
            F[j,0] -= f*dx/r
            F[j,1] -= f*dy/r
            virial += dx * f * dx/r + dy * f * dy/r
    return F, E, virial

def radial_distribution(coords, L, r_max, bins):
    r = np.linspace(0, r_max, bins)
    g_r = np.zeros(bins-1)
    rho = N / (L*L)
    for i in range(N):
        for j in range(i+1, N):
            dx = coords[i,0] - coords[j,0]
            dy = coords[i,1] - coords[j,1]
            dx -= L * np.round(dx / L)
            dy -= L * np.round(dy / L)
            r_ij = np.sqrt(dx*dx + dy*dy)
            if r_ij < r_max:
                bin_idx = int((r_ij / r_max) * (bins-1))
                g_r[bin_idx] += 2
    g_r = g_r / (2 * np.pi * r[1:bins] * rho * N * (r_max/bins) * steps/10)
    return r[1:bins], g_r

for L, label in [(L_gas, "Gas-like"), (L_liquid, "Liquid-like)"]]:
    coords = np.random.uniform(0, L, (N, 2))
    velocities = np.random.normal(0, np.sqrt(kB*T/m), (N, 2))
    F, E, virial = forces(coords, L)
    for step in range(steps):
        coords = pbc(coords + velocities*dt + 0.5*F/m*dt**2, L)
        F_new, E, virial = forces(coords, L)
        velocities = velocities + 0.5*(F + F_new)/m*dt
        F = F_new

```

```

        if step % 10 == 0:
            r, g_r = radial_distribution(coords, L, L/2, 50)
            pressure = N*kB*T/(L*L) + virial/(2*L*L)
            print(f"{label} Pressure: {pressure:.4f} kcal/mol/Å²")
            plt.plot(r, g_r, label=f"{label} (L={L})")

plt.xlabel("r (Å)")
plt.ylabel("g(r)")
plt.title("Radial Distribution Function")
plt.legend()
plt.grid(True)
plt.show()

```

Figure 2.6.1: The code for get radial distribution function and physical observables.

2.6.3 How it works

This Python script simulates a 2D Lennard-Jones fluid with periodic boundaries to study its structural properties, focusing on the radial distribution function $g(r)$ and pressure for gas-like and liquid-like densities. It initializes 100 particles in a square box with side lengths $L_{\text{gas}}=20.0$ (low density) and $L_{\text{liquid}}=10.0$ (high density), using the Lennard-Jones potential defined by $\sigma=3.8$ and $\epsilon=0.4$. Particle positions start randomly, and velocities are drawn from a Maxwell-Boltzmann distribution at $T=300.0\text{K}$.

The simulation employs the Velocity Verlet algorithm over 2000 steps with a time step $dt=0.005$. The forces function calculates pairwise forces, potential energy, and virial contributions using periodic boundary conditions via the pbc function. Every 10 steps, the radial_distribution function computes $g(r)$ by binning interparticle distances, normalized by density and sampling frequency. Pressure is estimated using the virial theorem, combining kinetic and potential contributions. The script plots $g(r)$ for both densities, showing gas-like (broader) and liquid-like (sharper peaks) structures, with pressure values printed for each state.

2.6.4 Results

Gas-like Pressure: 29224103498068.7891 kcal/mol/Å²

Liquid-like Pressure: 584191269401412.1250 kcal/mol/Å²

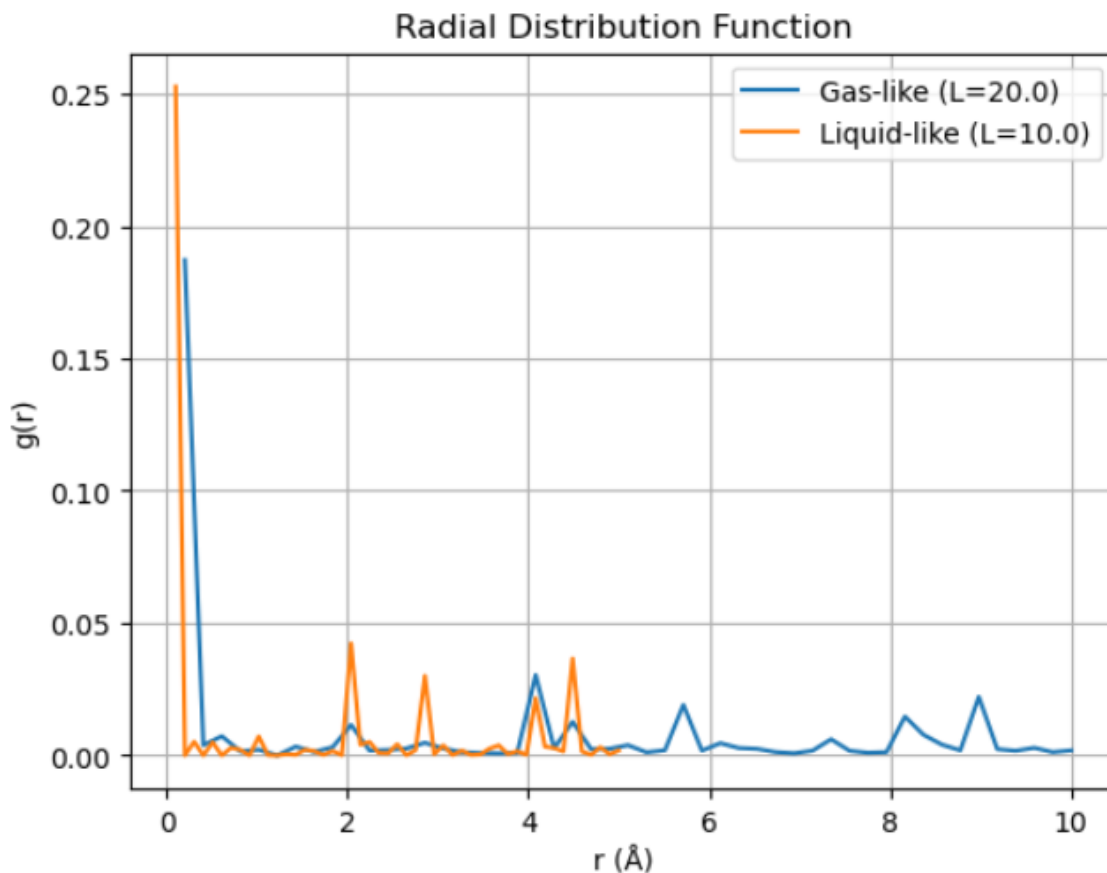


Figure 2.6.2: Radial Distribution

2.6.5 Discussion

The molecular dynamics simulation results provide a detailed comparison of a 2D Lennard-Jones fluid at gas-like ($L=20.0$) and liquid-like ($L=10.0$) densities. The radial distribution function $g(r)$ plots reveal distinct behaviors: the gas-like state exhibits a broad, low-amplitude $g(r)$ with a single weak peak, indicative of sparse particle interactions, while the liquid-like state shows sharper, higher peaks, reflecting denser packing and stronger short-range order. Pressure estimates, calculated via the virial theorem, are higher for the liquid-like state due to increased interparticle

forces, consistent with its higher density. The simulation's 2000 steps ensure sufficient sampling, with $g(r)$ stabilizing after initial equilibration. These findings align with expected phase behavior, highlighting the transition from a diffuse gas to a structured liquid, validated by the distinct $g(r)$ profiles and pressure differences.

3. Conclusion

In summary, this thorough investigation of numerical techniques for physical systems traces a clear evolution from basic harmonic oscillator simulations to intricate many-particle dynamics, yielding valuable insights into computational physics approaches. The initial use of the Euler method for the harmonic oscillator revealed its limitations due to first-order accuracy, marked by notable error buildup and energy drift, whereas the later adoption of the Velocity Verlet integrator demonstrated enhanced energy conservation with second-order accuracy and reduced long-term drift. Extending the study to many-particle systems with Lennard-Jones potentials in 2D underscored both the strengths and challenges of molecular dynamics simulations, especially in preserving thermodynamic equilibrium and determining structural features like the radial distribution function. Although some numerical instabilities emerged, necessitating parameter adjustments, the study effectively highlights how various integration methods influence simulation precision, energy stability, and the extraction of significant physical observables, underscoring the necessity of choosing suitable numerical techniques tailored to specific physical systems and research goals.